Utilizing Probabilistic Systems Analysis To Optimize Yahtzee Performance In Python

By: Doran Tavrow

Introduction

Yahtzee Rules

Probability Model for Five Dice
3.1 Combinatorial Structure of Dice Rolls
3.2 States and Multinomial Probability

Probability Model for Yahtzee
4.1 Category PMFs and Expected Value
4.2 Motivation for Dynamic Programming
4.3 Dynamic Programming Algorithm
4.4 DP-Derived PMFs

Yahtzee Simulation Framework
5.1 Implementation of the Yahtzee Class
5.2 Turn Structure and Decision Logic
5.3 Example DP Game Walkthrough

Results and Analysis
6.1 DP vs. Greedy Simulation Results
6.2 Analysis and Limitations

Conclusion

References

1. Introduction

Yahtzee is a classic dice game with a deceptively simple ruleset. A player rolls a set of 5 dice once, and then any subset or all 5 of them up to two more times a turn. At the end of each turn, they must fill one of 13 categories available with a score. Yahtzee can be interpreted as a stochastic dynamic system whereby strategic decision making after turn 1 and turn 2 in response to the outcome of a dice roll, or random variable, greatly influences success. Because the number of categories reduces each turn, the strategy is not only different roll by roll but also turn by turn. Decisions made earlier in the game can greatly impair the final score, making the application of probability theory particularly interesting.

This paper illustrates the use of dynamic programming to generate multinomial probability density functions for every category at every state, which is defined as the number of rolls remaining and dice output. It then utilizes a custom Yahtzee class to simulate tens of thousands of games by randomly generating dice rolls and navigating the number of dice to keep given the expected value of each category. This probability driven approach is compared to a greedy approach that does not forecast rolls and as such relies on shallow expected values. Following simulation the expected value and variance of the models is compared and outlier runs are analyzed to observe where the models fall short.

2. Yahtzee Rules

After a player has rolled 3 times or elects to stop early, one of the categories in the diagram to the left must be filled. Some categories require precise combinations of dice to score any points at all while others don't, some sum the values of the dice when a condition is met while others sum only some of them, and some categories give a single point value when achieved while others vary based on the dice values. We can see that each category in the 'upper section' of *Figure 2* takes the sum of all dice that show the correct number. '3 of a kind' and '4 of a kind' take the sum of all dice, requiring that three or four of the dice show the same number, respectively. Chance takes the sum of the dice and is treated as a backup for when a player has high scoring dice that don't fulfill another category. Small straight and large straight require 4 and 5 dice in a row respectively, a full house requires 3 of one type of dye and 2 of another, and yahtzee requires 5 dice all having the same value. If no categories can be fit by the dice after the 3rd roll, the player must cross off a category. There is an additional bonus provided to a player whose total upper score reaches or exceeds 63. Please note that there is no additional Yahtzee bonus, which is why these scores may appear lower than those found online, because the version of the game I am familiar with doesn't have one. As such online scores are not representative.

Upper Section	
Count and add only **Aces**	
Count and add only **Twos**	
Count and add only **Threes**	
Count and add only **Fours**	
Count and add only **Fives**	
Count and add only **Sixes**	
	Total
63 + scores a **35 Bonus**	
	Total of Upper Section

Lower Section	
3 of a kind	**Total of all dice**
4 of a kind	**Total of all dice**
Full House	**25**
Low Straight	**30**
High Straight	**40**
Yahtzee	**50**
Chance	**Total of all dice**
Yahtzee Bonus	**100**
	Total of Lower Section
	Total of Upper Section
	Grand Total

Figure 2: Yahtzee Scorecard

3. Probability Model for 5 Dice

3.1 Combinatorial Structure of Dice Rolls

It is important to recognize that not all rolls are created equal, or in this case with equal probability. This is because there are 6^5 = 7776 total ways to roll the dice when order is considered, but we only care about unique rolls.

Here we must introduce the idea of counts. For our purposes, a count is the number of each dice of each number. It makes up an array of length six where C_1 = number of dice valued 1 and so on. For the remainder of this paper, a dice roll is thought of and represented as a count:

Counts = $[C_1, C_2, C_3, C_4, C_5, C_6]$

Importantly, counts allow us to compute the number of unique rolls possible. It is best understood within the context of the binomial coefficient, formule provided in *Figure 3.1*, where:

- **n:** our space is the total number of counts and ways to separate the counts. Our counts sum to five and there are five ways to separate the array so we get that n = 10
- **k:** The total number of counts, which is five
- The total number of unique rolls can be solved as 10 choose 5 = 252

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Figure 3.1: Binomial Coefficient Equation

3.2 States and Multinomial Probability

In addition to recognizing that there is a space reduction from the number of total rolls to unique counts, we need to calculate the likelihood of achieving any given count from a roll. This is where multinomial probabilities, an extension of binomial probabilities, are necessary. A multinomial random variable[1] is defined as an array of length equal to the number of unique states, with each index containing the total number of observations of a given number of independent trials. The probability of rolling a given count 'c' is therefore calculated by the formula in *Figure 3.2*:

$$P(c_1, \ldots, c_6) = \frac{5!}{c_1!c_2!c_3!c_4!c_5!c_6!}\left(\frac{1}{6}\right)^5, \qquad \text{where } \sum_{i=1}^{6} c_i = 5.$$

Figure 3.2: Multinomial coefficient to calculate likelihood of rolling any given count

4 Probability Model For Yahtzee

4.1 Category PMFs and Expected Value

The probability mass function (PMF) for each category can be obtained by computing the score given by each of the 252 counts, then grouping by score and summing the probabilities of rolling each count that produces that score. The PMFs for various categories from a single roll are a good baseline, although the majority show a massive band at zero. Examples for each kind of scoring category are provided in *Figure 4.1*:

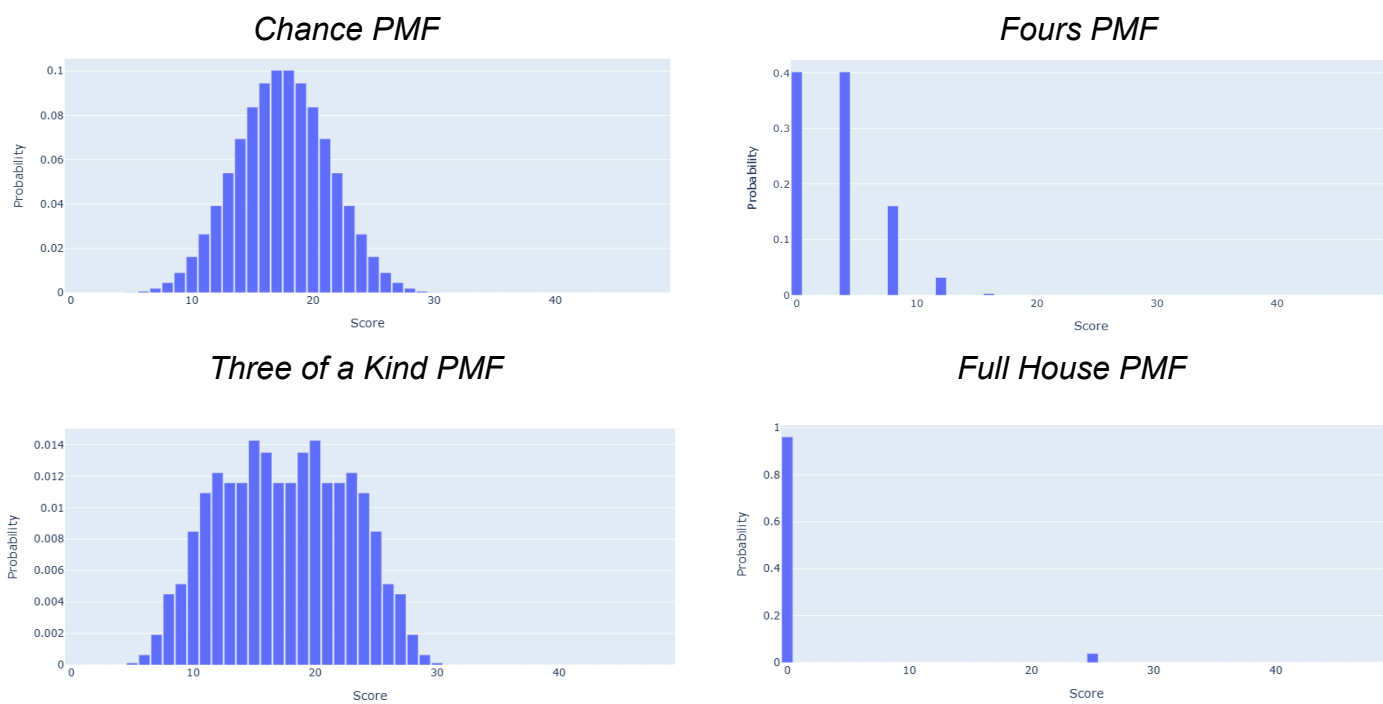

Figure 4.1: PMF examples (Note Y axis not to scale)

These represent the 4 types of PMFs that we see with Yahtzee. Chance has a gaussian distribution, full house represents the Bernoulli distributions, fours shows how all individual numbers look, and three of a kind showing a gaussian like centering around 2 points (excluding the 0.8 chance of scoring 0 that makes the graph unviewable). All other categories have one of these four types of distributions.

Without the need to go into detail, the expected value of all of these categories for a given roll, except chance, is obviously very low and therefore not all that useful. It is clear that to build a dynamic model we will need to take a more in-depth approach

4.2 Motivation for Dynamic Programming

The PMFs above are insufficient to model Yahtzee because they do not reflect what scores a player should expect to get when playing intelligently. With 3 turns, a player should be re-rolling to maximize their expected values across certain categories, of which this direction is based on the outcome of the first roll. As such we need to compute the optimal number of dice to reroll for each category given each initial state, which again factors in both the count and the number of rolls remaining.

For each state, we need to establish the action space, which reflects all actions that a person can take. This means first establishing all subsets of the count that can be rerolled, and then second computing a multinomial random variable for every reroll combination. This is done using the same approach as in *Section 3.2*, but this time for every possible new subcount Δ generated by rerolling a number d of the five dice:

$$\Delta = (\delta_1, \ldots, \delta_6), \qquad \sum_{i=1}^{6} \delta_i = d, \qquad P(\Delta) = \frac{d!}{\delta_1! \cdots \delta_6!} \left(\frac{1}{6}\right)^d$$

.

Figure 4.2: Multinomial probability for every reroll possibility given an initial state

Each of these subcounts are combined with the kept dice subcount to generate the next state. This problem utilizes a dynamic programing approach to memoize, or store, each of the possible 252 * 3 = 756 states for quick access and efficiency and because every state in a layer, r-1, can be reached by every state in the prior layer, r, as we allow the reroll of all five dice or none. But most importantly, because there is always the same optimal subsets of dye to keep and reroll for every initial state.

4.3 Dynamic Programming Algorithm

We define keep vectors k as an instance of the set of all subcounts K(c) that can be kept from a prior state. V(Δ + k, r) is then defined as an instance of all possible new states reachable by the data, where the new count c = V(c,r). Our goal with dynamic programming is to find the state V(c,r) that optimizes the expected value of the category F, so that we may keep the correct subset of dye and reroll the rest with the best chance of optimization. This gives us the recursive formula:

$$V(c,r) = \max_{k \in \mathcal{K}(c)} \sum_{\Delta} F(V(\Delta + k,\, r-1))P(\Delta)$$

Figure 4.3: DP Recursion Formula for any state V(c,r)

Informed by "Mathematical Formulation II - Discrete Stochastic Process" Section of Bellman et al.[2]

By generating the 256 x 3 matrix of states, the array of keeps for every possible count, and setting the base case V(c,0) = F(c), we have produced the dynamic programming algorithm callable by the function compute_pmf_for_state() in PMF_creator.ipynb. This returns a dictionary storing, for every state, the subcount Δ to keep and the PMF for the category given this Δ. V(c,r) is now unique for each (c,r) combination and advises the computer about how to proceed.

4.3 DP-Derived PMFs

The logical next step to take is to illustrate the new PMFs for each category at all initial states V(c,r=2):

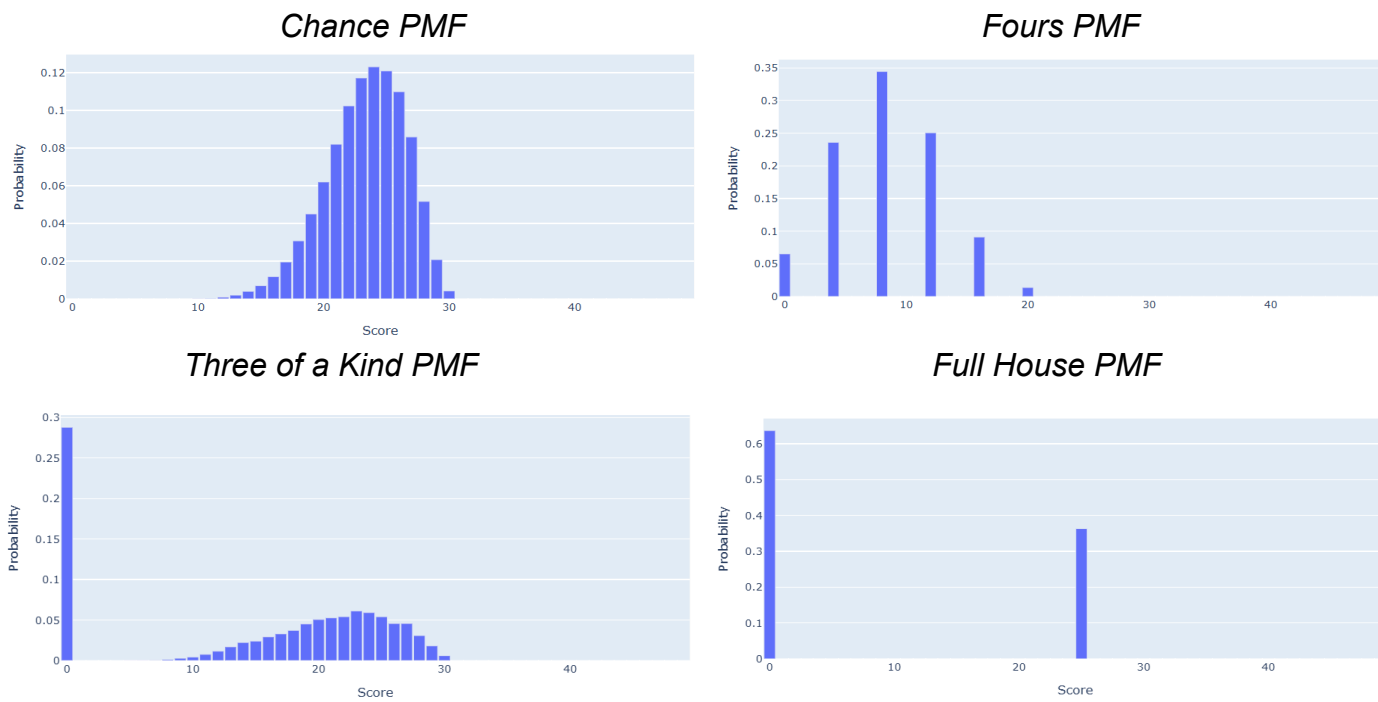

Figure 4.3: DP PMF Examples (Note Y axis not to scale)

Here we can see that the DP algorithm has completely transformed the PMFs of all of our variables. The expected value of chance is shifted from 17 to ~25, with 30 being the maximum possible. Fours has shifted so that the expected score is between two and three rolled dice. This is a key insight because the minimum score to achieve the bonus is reached when each category in the upper section has three dice per. We can see that the likelihood of achieving a non-zero score has jumped by ~50% and ~30% for three of a kind and full house respectively. Now we can proceed to simulating games.

5 Yahtzee Simulation

5.1 Implementation of the Yahtzee Class

The Yahtzee Simulator exists as class Yahtzee in Yahtzee.ipynb. It requires the V(c,r) matrices calculated for each category and their accompanying k. For each game, three turns are conducted. Np.random is used to generate random but reproducible results for every dice roll.

5.2 Turn Structure and Decision Logic

- At the start of turn 1, five dice are rolled and converted to a count c_2 and then the expected value of each category's PMF from $V(c_2,2)$ is calculated. The maximum expected value is selected to determine the new Δ, ending turn 1.
- At the start of turn 2, the number of dice designated by five - k = Δ are rolled and then combined with k to arrive at $V(c_1,1)$ and repeat the process of rolling the selected number of new dice.
- At the start of turn 3, the best score is selected from $V(c_0,0)$ and added to the composite score.

All available categories are listed as TRUE, and once a new score is added to the composite that category is marked as FALSE such that the selection of maximum expected score is made within the remaining open categories. Once 13 turns have passed, the bonus is applied if applicable and the total score is returned. No additional game theory or strategy is used, meaning that this game is played entirely through the lens of expected values.

5.3 Example DP Game Walkthrough

```
ROLL 1: (0.0, 2.0, 0.0, 2.0, 0.0, 1.0), Best Category: chance, Dice Kept: (0, 0, 0, 0, 0, 1)
ROLL 2: (3.0, 1.0, 0.0, 0.0, 0.0, 1.0), Best Category: chance, Dice Kept: (0, 0, 0, 0, 0, 1)
ROLL 3: (0.0, 0.0, 0.0, 2.0, 1.0, 2.0)
Best Category = chance, dice = (0.0, 0.0, 0.0, 2.0, 1.0, 2.0), Turn Score = 25, num turns = 1
ROLL 1: (0.0, 0.0, 0.0, 3.0, 1.0, 1.0), Best Category: three_of_a_kind, Dice Kept: (0, 0, 0, 3, 1, 1)
Best Category = three_of_a_kind, dice = (0.0, 0.0, 0.0, 3.0, 1.0, 1.0), Turn Score = 23, num turns = 2
ROLL 1: (0.0, 1.0, 0.0, 1.0, 2.0, 1.0), Best Category: small_straight, Dice Kept: (0, 0, 0, 1, 1, 1)
ROLL 2: (1.0, 0.0, 1.0, 1.0, 1.0, 1.0), Best Category: small_straight, Dice Kept: (1, 0, 1, 1, 1, 1)
Best Category = small_straight, dice = (1.0, 0.0, 1.0, 1.0, 1.0, 1.0), Turn Score = 30, num turns = 3
ROLL 1: (1.0, 0.0, 0.0, 1.0, 2.0, 1.0), Best Category: fives, Dice Kept: (0, 0, 0, 0, 2, 0)
ROLL 2: (1.0, 1.0, 0.0, 0.0, 2.0, 1.0), Best Category: fives, Dice Kept: (0, 0, 0, 0, 2, 0)
ROLL 3: (1.0, 0.0, 0.0, 1.0, 2.0, 1.0)
Best Category = fives, dice = (1.0, 0.0, 0.0, 1.0, 2.0, 1.0), Turn Score = 10, num turns = 4
ROLL 1: (1.0, 1.0, 3.0, 0.0, 0.0, 0.0), Best Category: threes, Dice Kept: (0, 0, 3, 0, 0, 0)
ROLL 2: (2.0, 0.0, 3.0, 0.0, 0.0, 0.0), Best Category: full_house, Dice Kept: (2, 0, 3, 0, 0, 0)
Best Category = full_house, dice = (2.0, 0.0, 3.0, 0.0, 0.0, 0.0), Turn Score = 25, num turns = 5
ROLL 1: (2.0, 0.0, 0.0, 2.0, 1.0, 0.0), Best Category: fours, Dice Kept: (0, 0, 0, 2, 0, 0)
ROLL 2: (0.0, 1.0, 0.0, 3.0, 1.0, 0.0), Best Category: fours, Dice Kept: (0, 0, 0, 3, 0, 0)
ROLL 3: (0.0, 0.0, 2.0, 3.0, 0.0, 0.0)
Best Category = fours, dice = (0.0, 0.0, 2.0, 3.0, 0.0, 0.0), Turn Score = 12, num turns = 6
ROLL 1: (0.0, 0.0, 1.0, 0.0, 2.0, 2.0), Best Category: sixes, Dice Kept: (0, 0, 0, 0, 0, 2)
ROLL 2: (0.0, 0.0, 0.0, 1.0, 0.0, 4.0), Best Category: four_of_a_kind, Dice Kept: (0, 0, 0, 1, 0, 4)
Best Category = four_of_a_kind, dice = (0.0, 0.0, 0.0, 1.0, 0.0, 4.0), Turn Score = 28, num turns = 7
ROLL 1: (1.0, 0.0, 1.0, 0.0, 2.0, 1.0), Best Category: sixes, Dice Kept: (0, 0, 0, 0, 0, 1)
ROLL 2: (0.0, 0.0, 0.0, 2.0, 1.0, 2.0), Best Category: sixes, Dice Kept: (0, 0, 0, 0, 0, 2)
ROLL 3: (0.0, 1.0, 2.0, 0.0, 0.0, 2.0)
Best Category = sixes, dice = (0.0, 1.0, 2.0, 0.0, 0.0, 2.0), Turn Score = 12, num turns = 8
ROLL 1: (1.0, 0.0, 1.0, 0.0, 1.0, 2.0), Best Category: large_straight, Dice Kept: (0, 0, 1, 0, 1, 0)
ROLL 2: (0.0, 0.0, 2.0, 1.0, 2.0, 0.0), Best Category: threes, Dice Kept: (0, 0, 2, 0, 0, 0)
ROLL 3: (0.0, 2.0, 2.0, 1.0, 0.0, 0.0)
Best Category = threes, dice = (0.0, 2.0, 2.0, 1.0, 0.0, 0.0), Turn Score = 6, num turns = 9
ROLL 1: (0.0, 0.0, 1.0, 2.0, 2.0, 0.0), Best Category: large_straight, Dice Kept: (0, 0, 1, 1, 1, 0)
ROLL 2: (0.0, 1.0, 2.0, 1.0, 1.0, 0.0), Best Category: large_straight, Dice Kept: (0, 1, 1, 1, 1, 0)
ROLL 3: (0.0, 1.0, 1.0, 1.0, 1.0, 1.0)
Best Category = large_straight, dice = (0.0, 1.0, 1.0, 1.0, 1.0, 1.0), Turn Score = 40, num turns = 10
ROLL 1: (1.0, 2.0, 0.0, 1.0, 1.0, 0.0), Best Category: twos, Dice Kept: (0, 2, 0, 0, 0, 0)
ROLL 2: (1.0, 2.0, 1.0, 0.0, 1.0, 0.0), Best Category: twos, Dice Kept: (0, 2, 0, 0, 0, 0)
ROLL 3: (0.0, 3.0, 1.0, 0.0, 1.0, 0.0)
Best Category = twos, dice = (0.0, 3.0, 1.0, 0.0, 1.0, 0.0), Turn Score = 6, num turns = 11
ROLL 1: (2.0, 1.0, 0.0, 0.0, 1.0, 1.0), Best Category: ones, Dice Kept: (2, 0, 0, 0, 0, 0)
ROLL 2: (2.0, 0.0, 1.0, 1.0, 1.0, 0.0), Best Category: ones, Dice Kept: (2, 0, 0, 0, 0, 0)
ROLL 3: (3.0, 0.0, 1.0, 0.0, 0.0, 1.0)
Best Category = ones, dice = (3.0, 0.0, 1.0, 0.0, 0.0, 1.0), Turn Score = 3, num turns = 12
ROLL 1: (0.0, 1.0, 0.0, 0.0, 3.0, 1.0), Best Category: yahtzee, Dice Kept: (0, 0, 0, 0, 3, 0)
ROLL 2: (1.0, 0.0, 0.0, 1.0, 3.0, 0.0), Best Category: yahtzee, Dice Kept: (0, 0, 0, 0, 3, 0)
ROLL 3: (0.0, 0.0, 1.0, 0.0, 3.0, 1.0)
Best Category = yahtzee, dice = (0.0, 0.0, 1.0, 0.0, 3.0, 1.0), Turn Score = 0, num turns = 13
Final score: 220
```

This output is generated from the first DP run, and shows the game decision making as it is guided by the PMFs of each category. For rolls one and two on each turn, we can see the initial roll, the expected value of the best category, and the k. After roll 3, or if a category with a Bernoulli PMF achieves a non-zero score, the turn is ended and we see the category that gains the turn score. Finally the program prints the final score.

Note that the model always succeeds at keeping the dice involved in the best category selection, and that this keep mechanism was calculated from all possible keeps as described in *Figure 4.2 and 4.3.* In addition, the model is capable of changing strategy when the second count has a maximum expected value for a different category. Take turn 9 when the count = (1,0,1,0,1,2) from turn 1 is rerolled to attempt to achieve a large straight. Only (0,0,1,0,1,0) are kept, seeing as they are the only requisite dice to achieve a large straight. A greedy algorithm or greedy individual may choose to keep (1,0,1,0,1) or even (1,0,1,0,1,2) thinking that they are close to a straight when expected value disagrees. The following count comes back (0,0,2,1,2), and with category fives filled already, it pivots to threes after deciding the expected value of achieving a large straight is now too low.

6.1 DP vs. Greedy Simulation Results

10,000 Yahtzee games were simulated for both the Greedy and DP state spaces with the following result in *Figure 5.4*, showing a clear separation in expected score between the two approaches.

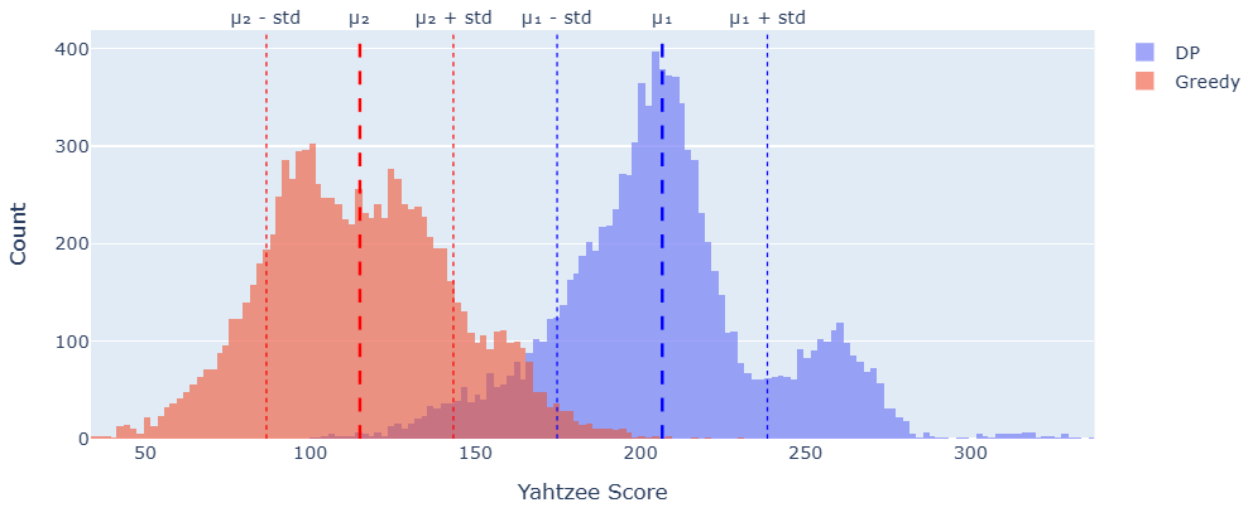

Figure 5.4: Yahtzee Simulation Results

- The expected value of the greedy solution is 115.0 compared to 206.6 for the DP solution.
- The standard deviations are 28.3 and 31.9 respectively.
- 99.9% of scores ≥ u_1 come from DP, where u_1 = 206.6
- 98.4% of scores ≥ u_1 - std from DP, where u_1 - std = 178.3
- 96.9% of scores ≤ u_2 come from greedy, where u_2 = 115.0
- 99.6% of scores ≤ u_2 + std come from greedy, where u_2 + std = 143.3

6.2 Analysis and Limitations

The DP approach has been shown to clearly, consistently, and intelligently surpass the greedy approach. In fact, online blogs and websites[3] seem to agree that a strong score with optimal strategy is 250, which is just over a standard deviation beyond the mean of my model. This means that ~15% of scores from this model are deemed strong, an impressive feat considering that little to no game strategy is applied.

However, it still fails spectacularly on some of the samples, scoring only slightly above 100 on multiple occasions. In analyzing most of the extreme outliers I found that they were caused by poor dice luck, which is amplified by a lack of game theory telling the game to cancel out the least likely categories of scoring. When no categories score points, it picks the first unfilled category to give a score of 0 in the dict.keys() where the keys output each category. This means that instead of canceling out the least likely categories that would likely improve overall score it does so at random. The model also doesn't take into account turn number, where an improved model may calculate variance to take more risks at the start of the game and prefer smaller variance near the end.

There is also a second peak ~260 that shows the results that managed to achieve the upper bonus and gain an additional 35 points. Because this bonus was not incorporated into the dynamic programming solution the expected values do not direct the model to modulate expected value in regards to how likely achieving the bonus may be. This explains why the peak appears gaussian, in that high scores will just happen to trip the bonus. If the bonus were not applied, we would see a more steady dropoff from $DP(u_1)$ to $DP(u_1 + std)$ and beyond.

7 Conclusion

By using dynamic programming to generate categorical PMFs selected in accordance with maximizing the expected value with respect to all possible states reachable on the next turn, we establish an optimal strategy at any state V(c,r) that guides the Yahtzee model to achieve a very solid score. By representing P(count | rolling 5 dice) with multinomial variables, my model accounts for the inherent stochasticity of the game, allowing the PMFs to hold up for the vast majority of simulations, far outpacing the greedy solution. With the incorporation of hard coded game strategy and the fixes to some of the limitations discussed above, I expect that this model would adequately compete with competent players.

This project taught me how to incorporate statistics into dynamic programming to simulate a system with entirely stochastic inputs. The back and forth between random events and logical decision making is very unique for a game, but highly applicable to more complex systems such as risk management in finance where decisions must constantly be made in response to random events. For a toy problem, it was a very valuable tool for reinforcing lessons of various random variable distributions, stochastic processes, and expected value taught in the class.

8 References

1. "Multinomial Probability Mass Function (PMF)." *Stat 2: A Modern Introduction to Probability and Statistics*, statproofbook.github.io, https://statproofbook.github.io/P/mult-pmf.html. Accessed 4 Dec. 2025.
2. Bellman, Richard. *The Theory of Dynamic Programming*. RAND Corporation, 30 July 1954. P-550
3. Playcards.co. "Yahtzee scoring guide: maximize your points every game." *PlayCards.co Blog*, 15 June 2025, https://playcards.co/blog/yahtzee-scoring-optimization/